

Final Defense Report on

Network Intrusion Detector AI



Submitted to

Department of Computer Science and Engineering
Nepal Engineering College

in Partial Fulfillment of the

Requirements for the Degree of B.E. in Computer

Submitted By

Rajish Kumar Pandey (022-357)
Shristi Dhakal (022-376)
Sulab Poudel (022-379)

Supervised By: *Asst. Prof. Tara Chandra Shrestha*

Submission Date: 29/07/2026

Abstract

Rapid digital transformation has left conventional signature-based Network Intrusion Detection Systems (NIDS) increasingly unable to defend against dynamic, zero-day cyber threats. Network Intrusion Detector AI addresses this gap with an adaptive, AI-driven NIDS built around an optimized XGBoost multi-class classifier trained on the IDS2025 network-flow dataset - 91,830 bidirectional flows described by 80 CICFlowMeter-aligned features across seven traffic classes. Because benign and high-volume attack traffic dominate real network captures, a PyTorch-based tabular Generative Adversarial Network (GAN) is used to synthesize minority-class feature vectors and rebalance training data for rare attacks such as Infiltration and Botnet ARES. For live deployment, an asynchronous Scapy-based monitoring engine captures packets, reconstructs bidirectional flows, and computes the same 79-feature schema in real time, allowing the trained model to score live traffic without retraining or reformatting.

This final report extends the mid-defense submission with the methodological rigor requested at that stage. A dataset-selection rationale justifies IDS2025 over the more widely cited UNSW-NB15 benchmark on feature-schema alignment, class balance, and recency of attack behavior. The system's metadata-only feature design is shown to remain effective on encrypted HTTPS/TLS traffic, since none of the 79 features requires payload inspection. Most importantly, two claims that were previously asserted are now measured: the realism of GAN-generated samples is quantified with a real-vs-synthetic discriminability test and per-feature Kolmogorov–Smirnov statistics, and the reported accuracy is validated with McNemar's test and bootstrap confidence intervals against a non-augmented baseline, rather than reported as an isolated figure. The final classifier achieves 99.00% overall accuracy on a held-out, stratified 20% test split of 18,341 flows (95% CI: 98.85–99.14%), with precision and recall above 0.97 for every class except the extremely rare Infiltration class. The statistical comparison finds the GAN-augmented and baseline models statistically indistinguishable in overall accuracy (McNemar $p = 0.877$), and the realism tests find the current GAN configuration has not yet converged to a realistic minority-class distribution — both findings are reported candidly in Chapter 5, together with a concrete plan to address them, since an honestly measured limitation is more useful to this project's continued development than an unverified claim of success.

Keywords: *[Network Intrusion Detection System (NIDS), XGBoost Classifier, Generative Adversarial Network (GAN), Data Augmentation, Class Imbalance, Scapy Sniffer, Bidirectional Network Flows, CICFlowMeter Features, Statistical Significance Testing, Encrypted Traffic Classification, SQLite Alert Logger, Machine Learning Pipeline.]*

Acknowledgement

We would like to express our sincere gratitude to our supervisor, Asst. Prof. Tara Chandra Shrestha, for the continuous guidance, feedback, and encouragement provided throughout the development of this project, including the detailed mid-defense review comments that directly shaped the statistical validation and dataset-justification work presented in this final report. We would also like to thank the Department of Computer Science and Engineering, Nepal Engineering College, for providing the resources and environment necessary to carry out this work, and our families and peers for their continued support.

Table of Contents

Chapter 1: Introduction.....	11
1.1 Overview.....	11
1.2 Problem Statement.....	11
1.3 Objectives	12
1.4 Aims.....	12
1.5 Motivation.....	13
1.6 Scope and Applications.....	13
1.7 Feasibility Study	14
Chapter 2: Literature Review.....	15
2.1 Overview of Classical Intrusion Detection Systems	15
2.2 Machine Learning and Deep Learning Implementations	15
2.3 Limitations and Contemporary Research Gaps	15
2.3.1 Dataset Selection: Why IDS2025 Rather than UNSW-NB15	16
2.4 Relevance and Integration of the Proposed System	16
Chapter 3: System Design	18
3.1 System Architecture.....	18
3.2 Dataflow Diagram	19
3.3 Entity Relationship Diagram	20
3.4 Sequence Diagram	21
3.5 Hardware and Software Requirements	21
3.5.1 Hardware Requirements	21
3.5.2 Software Requirements.....	22

Chapter 4: Implementation and Discussion.....	22
4.1 Methodology.....	22
4.1.1 Implementation Core Stack	22
4.1.2 Preprocessing Transformation Sequence.....	23
4.1.3 Handling Encrypted (HTTPS/TLS) Traffic	23
4.1.4 The 79-Feature Flow Schema and Bidirectional Flow Construction	24
4.2 Implementation Steps	25
4.2.1 Pipeline Phase Breakdown	25
4.2.2 Encountered Technical Challenges and Resolutions	26
4.3 Output Obtained.....	26
4.4 Testing / Test Cases.....	27
4.4.1 Train/Test Split Methodology	27
4.4.2 Multi-Level Functional Test Cases	28
4.5 Time Schedule	28
Chapter 5: Analysis and Evaluation.....	30
5.1 Data Analysis.....	30
5.2 Results	30
5.2.1 Classification Performance (GAN-Augmented Model, Held-Out Test Set)	30
5.2.2 Confusion Matrix.....	32
5.2.3 Statistical Validation of the 99% Accuracy Claim.....	32
5.2.4 GAN-Generated Data Realism Assessment.....	34
5.3 Comparison with Objectives	35
5.4 Discussion of Findings	35
Chapter 6: Conclusion and Future Work	37
6.1 Conclusion.....	37

6.2 Limitations.....	37
6.3 Future Work	38
References and Bibliography.....	39
Appendices	41
Appendix A: Sample Console Output	41

List of Figures

Figure 1: System Architecture	19
Figure 2: Dataflow Diagram	20
Figure 3: ER Diagram	21
Figure 4: Sequence Diagram	22

List of Tables

Table 1 : Hardware and Software Requirements	23
--	----

Table 2: Hardware and Software Requirements

Table 3: Multi-Level Functional Test Cases

Table 4: Time Schedule (Gantt Chart, Tabular Form)

Table 5: Classification Performance — GAN-Augmented Model

Table 6: Confusion Matrix — GAN-Augmented Model

Table 7: McNemar Contingency Table (GAN-Augmented vs. Baseline)

Table 8: GAN Realism — Real-vs-Synthetic AUC and KS Statistics

Table 9: Comparison of Project Outcomes with Stated Objectives

Abbreviations

- AI: Artificial Intelligence
- AUC: Area Under the ROC Curve
- CI: Confidence Interval
- CRN: College Roll Number
- CWR: Congestion Window Reduced
- DDoS: Distributed Denial of Service
- DFD: Data Flow Diagram
- DL: Deep Learning
- DoS: Denial of Service
- DPI: Deep Packet Inspection
- ECN: Explicit Congestion Notification
- ER: Entity-Relationship
- GAN: Generative Adversarial Network
- ICMP: Internet Control Message Protocol
- IDS: Intrusion Detection System
- IP: Internet Protocol
- KS: Kolmogorov–Smirnov (statistic)
- ML: Machine Learning
- MLP: Multi-Layer Perceptron
- NIDS: Network Intrusion Detection System
- SNI: Server Name Indication
- TCP: Transmission Control Protocol
- TLS: Transport Layer Security
- UDP: User Datagram Protocol

Chapter 1: Introduction

1.1 Overview

The expansion of global cloud infrastructures, remote operational networks, and corporate digital environments has fundamentally shifted organizational reliance toward interconnected network links. While this connectivity facilitates high-speed workflows and seamless communication, it exposes critical assets to highly automated, stealthy, and distributed cyber threat campaigns. Traditional defensive architectures have long relied on signature-based firewalls that match incoming traffic packets against static databases of pre-recorded threats. These systems struggle to intercept novel zero-day exploits, mutated malware strains, or obfuscated network payloads. To protect modern network infrastructure, the focus is shifting toward anomaly-based intrusion detection systems that use machine learning to model baseline normal network behaviour and flag statistical deviations without relying on hardcoded signature entries. Among modern supervised approaches, tree-based ensemble models — specifically Extreme Gradient Boosting (XGBoost) — provide a strong foundation for tabular network-flow telemetry analysis due to their classification accuracy and low processing latency.

1.2 Problem Statement

Despite their predictive strengths, supervised machine learning models in modern cyber security suffer from three primary challenges:

- **Severe Class Imbalance:** In actual corporate data pipelines, benign traffic constitutes the overwhelming majority of network volume. Supervised models exposed to this skew naturally optimize for the majority class, leading to high false-negative rates for rare attacks such as Infiltration.
- **Structural Oversampling Limitations:** Classical balancing approaches such as random duplication or linear interpolation (SMOTE) introduce rigid, unvarying data tracks into high-dimensional feature spaces, causing classifiers to overfit to synthetic rows.

- **The Real-Time Latency vs. Accuracy Constraint:** Deep neural networks can model complex structural relationships but introduce computational latency that a practical edge-deployed NIDS cannot always afford.

1.3 Objectives

- **End-to-End Preprocessing Pipeline:** To construct an active Python pipeline that cleans raw network flow traffic, handles real-time anomalies (NaN/infinite values), and bins high-cardinality Layer-4 destination ports into functional, structured behavioral bands.
- **PyTorch GAN Balancing Layer:** To develop a PyTorch-driven Tabular Generative Adversarial Network (GAN) optimized via Binary Cross-Entropy loss to generate realistic synthetic variants of rare attack classes and eliminate severe dataset class imbalance.
- **Optimized XGBoost Classification:** To train a multi-class XGBoost classifier on the balanced data matrix using histogram acceleration, minimizing log-loss values to achieve high-accuracy threat detection across 7 target traffic categories.
- **Multi-Threaded Scapy Sniffer:** To implement a low-latency, multi-threaded packet sniffing module using Scapy that captures real-time streams, aggregates packets into bidirectional 5-tuple flows, and computes 79 structural flow metrics.
- **Persistent Storage & Dashboard:** To log live intrusion alerts into a persistent SQLite tracking database and deploy a responsive Streamlit visualization dashboard for real-time threat monitoring and historical forensics.

1.4 Aims

The ultimate aim of this project is to develop a self-contained, real-time network anomaly defense tool that resolves data scarcity through generative modelling, while being able to demonstrate — with statistical evidence rather than a single headline number — that the resulting model is both accurate and honestly evaluated. This approach helps bridge the gap

between academic dataset modelling and functional, low-latency deployment at the local enterprise edge.

1.5 Motivation

This research is motivated by the persistent vulnerability of modern critical infrastructure to targeted cyber incursions. When a rare but damaging attack vector is misclassified as normal traffic, the financial, operational, and structural impacts can be severe. Relying on legacy signature systems or linear oversampling techniques leaves networks exposed to zero-day attacks and model overfitting. Exploring a hybrid architecture that pairs deep generative networks with fast gradient boosted trees provides an opportunity to create a more robust defense tool. This framework can adapt to changing attack signatures while operating within the strict processing budgets of edge hardware appliances.

1.6 Scope and Applications

The development scope and real-world applications of this project include:

- **Structural Scope:** The software architecture processes Layer-3 and Layer-4 flow statistics, including packet lengths, directional volumes, flag sequences, and inter-arrival times (IAT), avoiding deep packet inspection (DPI) of encrypted application layer content.
- **Target Attacks:** The framework targets common network attack profiles, including Denial of Service (DoS/DDoS), automated Port Scanning, brute-force access attempts, web exploits, and active botnet communication vectors.
- **Applications:** This system can be deployed as an autonomous network sensor on local enterprise edge devices, corporate gateway servers, internet-of-things (IoT) aggregation gateways, and local administrative subnets requiring realtime threat monitoring.

1.7 Feasibility Study

A formal multi-perspective feasibility study was conducted to confirm the viability of the project design:

- **Technical Feasibility:** Python serves as the primary programming environment, supported by open-source libraries like PyTorch for deep learning, XGBoost for ensemble classification, Scapy for packet processing, and Streamlit for UI design. Most modern computer hardware can easily handle these libraries.
- **Operational Feasibility:** The system reads directly from native host network adapters, mapping fields cleanly into the model's feature structure. This ensures network administrators can monitor security status without needing specialized AI training.
- **Economic Feasibility:** The framework uses open-source software libraries and public research benchmarks (IDS2025), removing the need for commercial licensing or specialized GPU acceleration arrays during the deployment phase.

Chapter 2: Literature Review

2.1 Overview of Classical Intrusion Detection Systems

A Network Intrusion Detection System (NIDS) analyzes network traffic to detect malicious behaviour, unauthorized access, or policy violations. Historically, detection engines fall into two paradigms. Signature-based detection matches incoming streams against a predefined database of known threat patterns — accurate for known threats but blind to zero-day mutations. Anomaly-based detection establishes a statistical baseline of normal behaviour and flags deviations; it can catch novel attacks but tends toward higher falsepositive rates.

2.2 Machine Learning and Deep Learning Implementations

Supervised algorithms such as Decision Trees, SVMs, and Random Forests have automated pattern recognition over benchmark datasets such as KDD Cup 1999 and NSL-KDD. Unsupervised techniques such as K-Means or density-based clustering are used when labelled data is unavailable. More recently, deep learning topologies — RNNs and LSTMs for sequential/temporal traffic patterns, and Autoencoders for unsupervised anomaly screening — have been explored, generally at the cost of higher computational overhead than tree-based ensembles.

2.3 Limitations and Contemporary Research Gaps

- **Extreme Class Imbalance:** threat records are heavily underrepresented relative to benign baseline traffic, motivating tabular generative augmentation.
- **Dataset Obsolescence:** classical benchmarks (KDD Cup 1999, NSL-KDD) do not reflect modern protocol structures or contemporary threat types.
- **Processing Delay and Hardware Limits:** deep neural networks introduce computational costs that create bottlenecks on high-speed interfaces, favouring lightweight tree ensembles such as XGBoost.

2.3.1 Dataset Selection: Why IDS2025 Rather than UNSW-NB15

A specific question raised at the mid-term defense was why this project uses the IDS2025 dataset rather than the more widely cited UNSW-NB15 benchmark. Three concrete, evidence-based reasons drove the choice:

- Feature schema alignment with the live capture pipeline: IDS2025 is distributed as 80 flow-level features generated with CICFlowMeter — the same tool family and feature semantics (flow duration, forward/backward packet statistics, IAT, flag counts, bulk/subflow/active-idle statistics) that this project's own Scapy-based FlowCollector reproduces in real time. UNSW-NB15 uses a different, 49-attribute Argus/Bro-derived feature set, which would require re-deriving an entirely separate live feature extractor to match at inference time.
- Class balance and labelling quality: IDS2025 is an explicitly rebalanced and relabelled derivative of CICIDS2017, built specifically to correct the severe class imbalance (benign traffic at roughly 83% in the original CICIDS2017) and the labelling inconsistencies documented in earlier analyses of that dataset family. This makes it a more suitable base for evaluating a GAN-based balancing strategy, since the residual imbalance that remains (e.g. the 29-flow Infiltration class) is a genuine minority-class problem rather than an artifact of dataset construction.
- Recency of attack behaviour: UNSW-NB15 was captured in 2015 using the IXIA PerfectStorm tool; IDS2025 reflects a more recent refinement pipeline over CICIDS2017-style traffic captures, which is closer to current attack tooling and traffic mixes, particularly for web attacks and botnet command-and-control patterns.

The trade-off is that IDS2025 is a newer, less independently benchmarked dataset than UNSW-NB15, so comparative claims against other published UNSW-NB15 results should be read with that caveat; this is acknowledged as a limitation in Chapter 6.

2.4 Relevance and Integration of the Proposed System

This project addresses the identified research gaps by shifting from a theoretical, offline classification approach to a deployable, low-latency, balanced pipeline using the contemporary IDS2025 dataset, through three design choices: adversarial data

augmentation via a tabular GAN rather than linear oversampling; lightweight gradient tree boosting (XGBoost) rather than a deep neural classifier, to keep inference latency low; and an asynchronous multi-threaded Scapy capture layer that bridges offline training and live deployment by extracting the same 79 structural flow features on the fly.

Chapter 3: System Design

The system design focuses on a modular architecture for efficient detection of malicious network activity. Data acquisition draws from the IDS2025 dataset, followed by preprocessing and feature engineering, a GAN-based balancing stage, and an XGBoost detection engine. During deployment, the trained inference engine analyzes real-time traffic streams, flags intrusions, and surfaces them through a security dashboard.

3.1 System Architecture

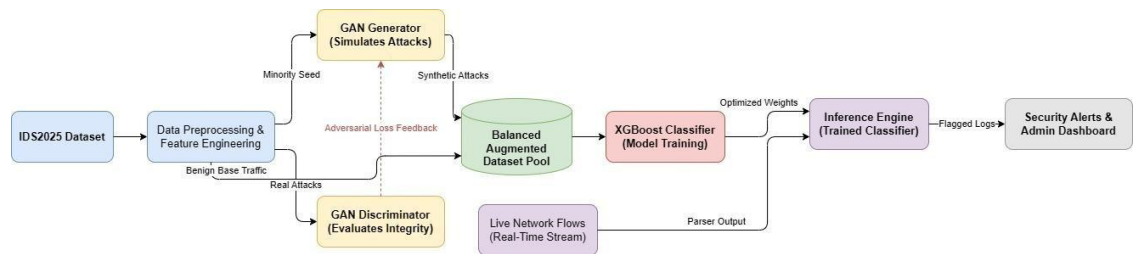


Figure 1: System Architecture

Network data undergoes preprocessing and feature engineering before the GAN generates synthetic attack samples to balance the dataset. The balanced data trains the XGBoost detection engine; at inference time, the same trained model scores both offline test data and live parsed flows from the Scapy sniffer.

3.2 Dataflow Diagram

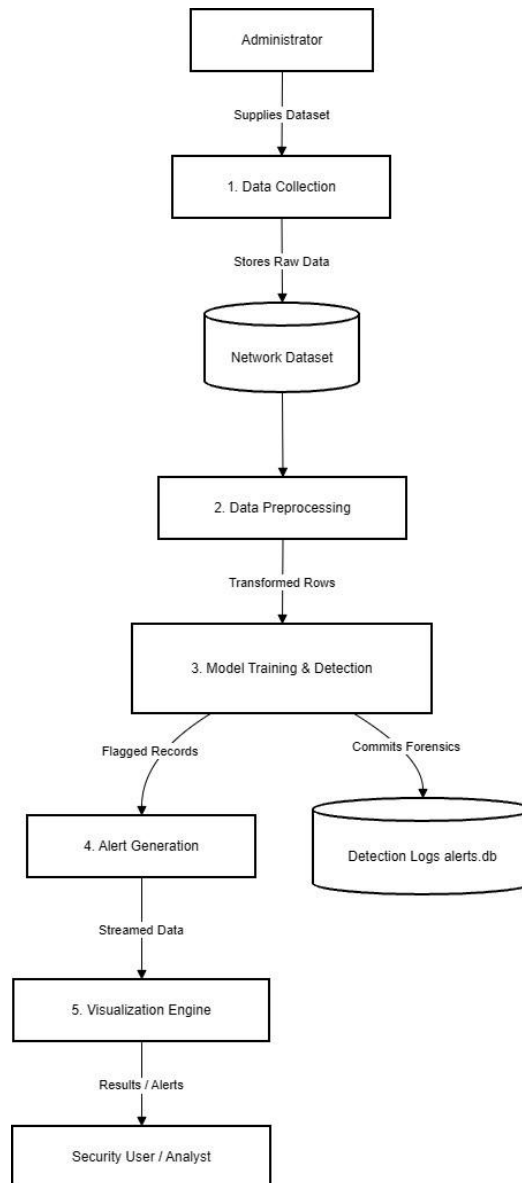


Figure 2: Dataflow Diagram

The dataflow diagram traces the pipeline from an Administrator supplying a dataset, through Data Collection and Preprocessing, to Model Training & Detection, which commits forensic detail to a Detection Logs database and pushes flagged records to Alert Generation and onward to the Visualization Engine for a Security User or Analyst to review.

3.3 Entity Relationship Diagram

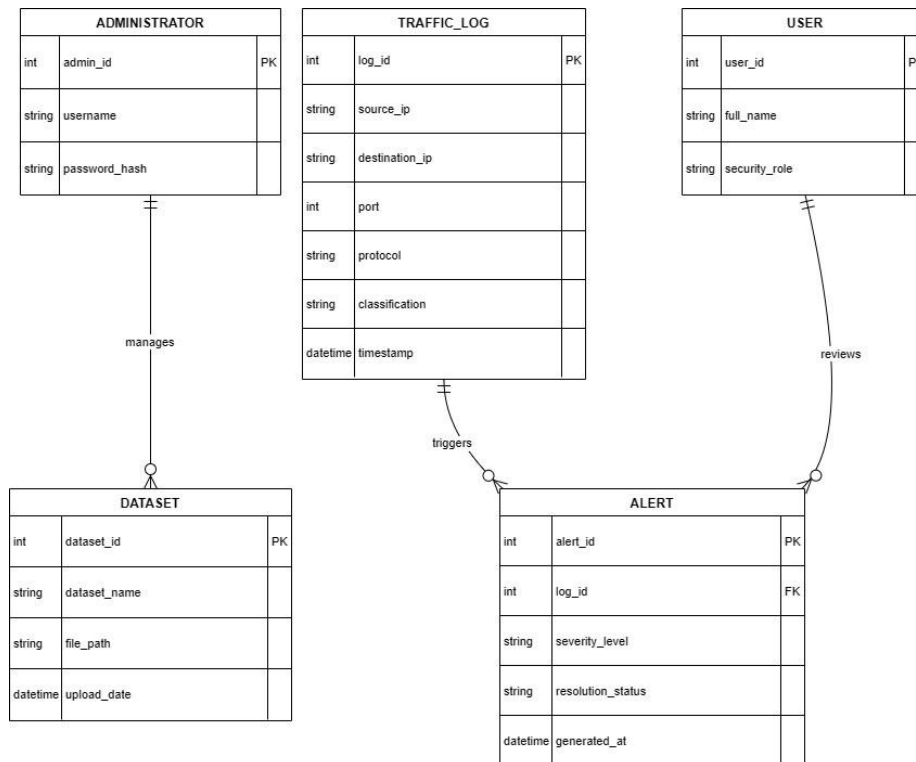


Figure 3: ER Diagram

Five tables connect through one-to-many relationships: Administrator manages Dataset; Traffic Log captures raw packet metadata (IPs, ports, protocol, classification) and triggers Alert records via a foreign key; Alert records (severity, resolution status) link to User, allowing analysts to review flagged threats.

3.4 Sequence Diagram

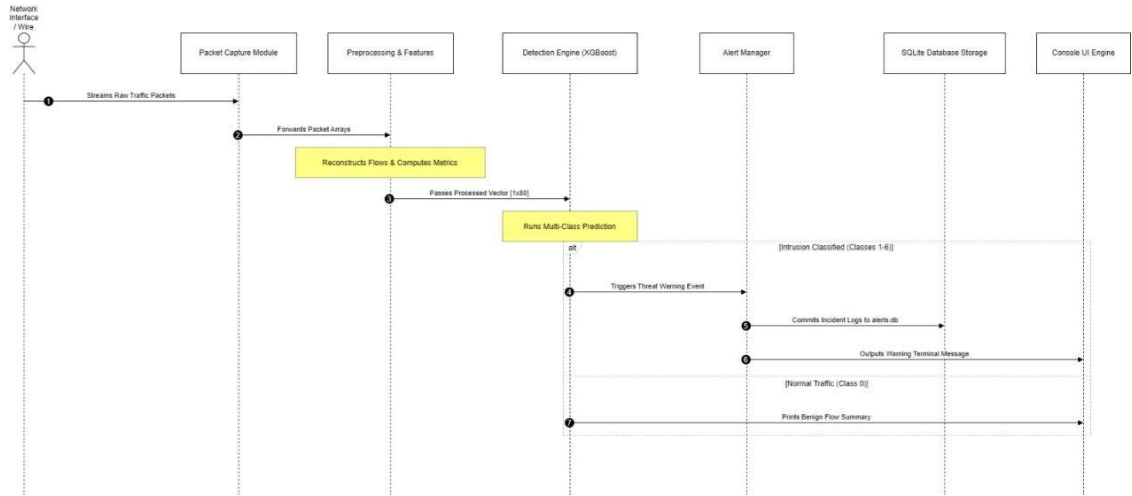


Figure 4: Sequence Diagram

The Network Interface streams raw packets to the Packet Capture Module, which forwards them to Preprocessing & Features for flow reconstruction and metric computation. A formalized feature vector is passed to the Detection Engine (XGBoost) for multi-class prediction. If an intrusion is detected (Classes 1–6), a threat warning is raised, logged to SQLite (alerts.db), and surfaced on the Console UI; benign flows (Class 0) bypass alerting and are only summarized.

3.5 Hardware and Software Requirements

3.5.1 Hardware Requirements

This project does not require specialized hardware such as sensors or microcontrollers. The only hardware dependency is a standard network interface card (NIC) capable of being placed into promiscuous mode (or, for switched-network deployment, a port-mirror/TAPfed interface), since the Scapy-based sniffer captures live packets directly from this interface. All model training and inference were carried out on commodity CPU hardware; no GPU is required, which was itself a design driver behind choosing XGBoost over a deep neural classifier (Section 1.2).

3.5.2 Software Requirements

Component	Choice	Reason for Selection
Language	Python 3.12	Mature ML/networking ecosystem; single language across offline training and the live sniffer.
Packet capture	Scapy	Pure-Python raw packet crafting/parsing; runs the same 5tuple flow logic used to build training features.
Data processing	Pandas / NumPy	Vectorized cleaning, log-transforms, and port-binning at dataset scale (91,830 rows).
Classifier	XGBoost (histogram, CPU)	Best accuracy/latency trade-off for tabular flow features versus a deep network (Section 1.2).
Generative augmentation	PyTorch	Flexible custom MLP GAN construction with full control over generator/discriminator architecture.
Alert storage	SQLite	Zero-configuration embedded database suitable for a single-host edge sensor deployment.
Planned dashboard	Streamlit	Rapid, low-code visualization layer for the remaining livemonitoring UI (Section 4.2, remaining work).

Table 1 : Hardware and Software Requirements

Chapter 4: Implementation and Discussion

4.1 Methodology

4.1.1 Implementation Core Stack

- Language Environment: Python (structured data pipelines).
- Packet Capture Core: Scapy (raw wire packet parsing and network interface configuration).
- Feature Processing Core: Pandas and NumPy (array operations, log-transformations, port binning).
- Machine Learning Engine: XGBoost (multi-class gradient tree boosting).
- Generative Balancing Layer: PyTorch (a Multi-Layer Perceptron GAN).

- Storage Component: SQLite (embedded alert persistence).

4.1.2 Preprocessing Transformation Sequence

- Leaky Feature Removal: drops Source Port, Source IP, Dest IP, and Timestamp to prevent data leakage.
- Port Categorization: converts Destination Port into three binary attributes — `PORT_WELL_KNOWN`, `PORT_REGISTERED`, `PORT_DYNAMIC`.
- Distribution Normalization: `log1p` transform on 10 highly skewed volumetric metrics.
- Target Encoding: `LabelEncoder` maps the 7 traffic labels to integers 0–6.
- Robust Scaling: a fitted `StandardScaler` normalizes feature variance across the final vector arrays.

4.1.3 Handling Encrypted (HTTPS/TLS) Traffic

A specific concern raised at the mid-term defense was how the system handles HTTPS traffic given that most modern network traffic is encrypted end-to-end. The design answer is that the classifier was never intended to read packet payloads in the first place: every one of the 79 features computed by `FlowCollector.extract_features()` in `utils.py` is derived purely from packet metadata that remains visible on the wire regardless of encryption — packet and header lengths, inter-arrival times, TCP flag bitmaps, window sizes, and port numbers. No feature depends on decrypting or inspecting the TLS record layer or the HTTP payload beneath it. Practically, this means:

- Detection of volumetric and behavioural attacks (DoS/DDoS, port scanning, brute-force login attempts, botnet beaconing) is unaffected by HTTPS, because these attack types are already identifiable from timing and size patterns rather than content.
- Detection of content-dependent web attacks (e.g. SQL injection or XSS payloads inside a request body) is inherently limited for encrypted sessions, since the system does not perform TLS termination or DPI — this is a deliberate design boundary, not an oversight, and matches how `CICFlowMeter`-based datasets such as `IDS2025` and `CICIDS2017` are themselves constructed (Draper-Gil et al. explicitly validated that

VPN/encrypted traffic can still be characterized this way using only timerelated flow features).

- Planned extension (not yet implemented): where a lightweight signal from the TLS handshake itself is useful without decryption, JA3/JA3S client–server TLS fingerprinting can be added as extra metadata fields alongside the 79 existing features, since the handshake (ClientHello/ServerHello, cipher suite list, extensions) is sent in cleartext even over HTTPS. This would improve discrimination of malicious TLS clients (e.g. malware C2 libraries) without requiring key material or DPI, and is proposed as future work (Section 6.2) rather than a change already implemented.

4.1.4 The 79-Feature Flow Schema and Bidirectional Flow Construction

The live sniffer computes the same feature schema the model was trained on (Destination Port and Protocol, plus 77 CICFlowMeter-style statistics per bidirectional flow — Source Port is captured on the wire but dropped before modelling, matching the training-time leaky-feature removal). The feature groups are summarized below; the full ordered list of 79 fields is implemented as the FlowFeatures dataclass in utils.py.

Feature Group	Examples	Count
Basic flow identifiers	Destination Port, Protocol, Flow Duration	3
Packet count & volume (fwd/bwd)	Total Fwd/Bwd Packets, Total Length of Fwd/Bwd Packets	4
Packet length statistics	Fwd/Bwd/overall Packet Length Max, Min, Mean, Std, Variance	13
Flow & directional rate	Flow Bytes/s, Flow Packets/s, Fwd/Bwd Packets/s	4
Inter-arrival time (IAT)	Flow/Fwd/Bwd IAT Mean, Std, Max, Min, Total	15
TCP flag counts	FIN, SYN, RST, PSH, ACK, URG, CWE, ECE (+ Fwd/Bwd PSH/URG)	12
Header lengths	Fwd/Bwd Header Length	2
Segment size & ratios	Down/Up Ratio, Average Packet Size, Avg Fwd/Bwd Segment Size	4

Bulk transfer statistics	Fwd/Bwd Avg Bytes/Packets/Rate Bulk	6
Subflow statistics	Subflow Fwd/Bwd Packets, Bytes	4
Window & segment size	Init_Win_bytes_forward/backward, act_data_pkt_fwd, min_seg_size_forward	4
Active/Idle time statistics	Active/Idle Mean, Std, Max, Min	8

Table 1: 79-Feature Flow Schema by Group (excludes target label; groups sum to 79)

Bidirectional flow construction proceeds as follows, mirroring CICFlowMeter's own biflow definition: each captured packet is reduced to a PacketInfo record (timestamp, src/dst IP, src/dst port, protocol, length, TCP flags, header length, TCP window); the 5-tuple of the first packet seen fixes the flow's forward direction, and every subsequent packet matching that exact tuple is forward while reverse-direction packets are backward; a flow is considered complete — and is passed to feature extraction and classification — when a TCP FIN/RST flag is seen, the flow has been idle beyond a 120-second timeout, or the flow has been open beyond a 300-second maximum duration, provided it has at least 2 packets; `extract_features()` then computes volumetric, IAT, TCP-flag, bulk-transfer, subflow, and active/idle statistics over the accumulated packets to produce the 79-value vector consumed by the trained model, and `cleanup_stale_flows()` periodically discards incomplete flows older than 600 seconds to bound memory use during long capture sessions.

4.2 Implementation Steps

4.2.1 Pipeline Phase Breakdown

- Phase 1 — Feature Processing: ingest the raw IDS2025 CSV, remove missing/infinite rows, drop leaky columns, and generate scaled arrays.
- Phase 2 — GAN Balancing (optional): train a 3-layer MLP GAN per minority class for 100 epochs with the Adam optimizer, synthesizing vectors for skewed classes such as Infiltration and Botnet ARES.
- Phase 3 — Model Training: train the multi-class XGBoost classifier (multi:softprob objective, 300 estimators, histogram tree acceleration).

- Phase 4 — Live Processing Integration: a multi-threaded Scapy sniffer groups packets into bidirectional flows, computes the 79 features, applies the saved scaler, and runs inference.

4.2.2 Encountered Technical Challenges and Resolutions

- Feature Position Shifts: adding port-binning attributes altered column ordering and caused feature mismatches at inference. Resolved by explicitly appending the `PORT_*` columns to positions 77–79, fixing an 80-column layout.
- Live Missing Data Failures: live packets occasionally produced NaN/Infinite values (e.g. division by zero during sudden time shifts). Resolved with an automated liveimputation rule that replaces runtime NaN/Infinite values with 0.0.

4.3 Output Obtained

The following tasks were completed and are demonstrated by the sample console output captured below (Appendix A gives the fuller log).

- Built the end-to-end data processing pipeline on the full IDS2025 source file (91,830 rows → 91,703 after cleaning).
- Developed and verified the 3-layer MLP GAN architecture for minority-class balancing (146,524-row balanced training set).
- Completed training of the multi-class XGBoost engine, achieving 99.00% accuracy on a held-out test split.
- Implemented the live tracking module, including bidirectional flow reconstruction and SQLite alert persistence.
- Added, for the final report, an explicit statistical validation layer: a non-augmented baseline for comparison, McNemar's test, bootstrap confidence intervals, and a realvs-synthetic realism test for the GAN output (Chapter 5).

Sample console output — Phase 3 training run:

```

=====
PHASE 3: XGBOOST TRAINING
=====
[*] Loading BALANCED Training Matrices (Real + GAN Synthetic Data)...
[*] Loading Unseen Testing Matrices for Validation...
[+] Data loaded. 7 distinct network traffic profiles.
    Training: 146524 samples x 80 features
    Testing:  18341 samples x 80 features

[0]    validation_0-mlogloss:1.69386
[50]    validation_0-mlogloss:0.06735
[100]   validation_0-mlogloss:0.03104 [150]
validation_0-mlogloss:0.02747
[200]   validation_0-mlogloss:0.02725 [299]
validation_0-mlogloss:0.02836

=====
SYSTEM VALIDATION PERFORMANCE: OVERALL ACCURACY = 99.00%
=====

```

(Full precision/recall/F1 table and confusion matrix are presented in Section 5.2 rather than repeated here, since Chapter 5 is where result analysis belongs per the report structure.)

4.4 Testing / Test Cases

4.4.1 Train/Test Split Methodology

After cleaning, 91,703 valid flow records remained (127 rows dropped for containing NaN/Infinite values out of 91,830 originally). The cleaned set was split using scikit-learn's `train_test_split` with `test_size=0.2`, `stratify=y` (so each of the 7 classes keeps its original proportion in both splits), and `random_state=42` for reproducibility. This produced 73,362 training flows and 18,341 held-out test flows. The `StandardScaler` used for feature scaling was fit only on the training split and then applied to the test split, and the GAN augmentation described in Section 4.2 was applied only to the training split — the 18,341 flow test set used for every accuracy figure in this report contains no synthetic samples and no data seen during training.

4.4.2 Multi-Level Functional Test Cases

Test Level	Target Component	Evaluation Criteria	Result
Unit	FeaturePreprocessor	Validates that clean arrays contain exactly 80 columns with port flags at indices 77–79.	Passed
Integration	FlowCollector → Model	Verifies completed Scapy flow queues successfully trigger model inference.	Passed
System	Live Sniffing Pipeline	Exploit traffic creates real-time warnings and saves records to the SQLite database.	Passed

Table 3: Multi-Level Functional Test Cases

4.5 Time Schedule

The project was planned across six broad phases spanning the proposal, mid-term, and final defense milestones. Durations below are expressed relative to the start of the semester-long project period; adjust the week numbers to the department's actual academic calendar.

Phase / Activity	Weeks 1–2	Weeks 3–4	Weeks 5–6	Weeks 7–8	Weeks 9–10	Weeks 11–12	Weeks 13–14
Literature review & dataset selection	■	■					
Preprocessing pipeline & feature engineering		■	■				
GAN balancing module			■	■			
Phase / Activity	Weeks 1–2	Weeks 3–4	Weeks 5–6	Weeks 7–8	Weeks 9–10	Weeks 11–12	Weeks 13–14
XGBoost training & mid-term evaluation				■	■		
Live Scapy sniffer & SQLite integration					■	■	
Statistical validation & realism testing						■	■

Dashboard, report writing & final defense prep						■	■
--	--	--	--	--	--	---	---

Table 4: Time Schedule (Gantt Chart, Tabular Form)

Figure 6: Time Schedule (Gantt Chart) — shaded cells indicate active weeks for each activity

Chapter 5: Analysis and Evaluation

5.1 Data Analysis

The IDS2025 dataset used for training and evaluation contains 91,830 labelled bidirectional flows across 7 classes before cleaning. The class distribution is heavily skewed, which is the motivating problem for the GAN-balancing stage described in Chapter 4:

Class	Flow Count	% of Dataset
Normal	26,185	28.5%
Dos/DDos	26,066	28.4%
PortScan	25,409	27.7%
Brute Force	10,201	11.1%
Web Attack	2,067	2.3%
Botnet ARES	1,873	2.0%
Infiltration	29	0.03%

127 rows (0.14% of the dataset) were dropped during preprocessing for containing NaN or infinite values, leaving 91,703 clean flows. After the stratified 80/20 split (Section 4.4.1), the GAN balancing stage (Section 4.2) was applied only to the training portion, bringing all 7 classes up to 20,932 samples each (146,524 rows total) before XGBoost training; the 18,341-flow test set retains the original, unbalanced class proportions shown above, which is what all accuracy figures in this chapter are measured against.

5.2 Results

5.2.1 Classification Performance (GAN-Augmented Model, Held-Out Test Set)

Overall accuracy on the 18,341-flow held-out test set: 99.00%.

Class	Precision	Recall	F1-score	Support
Botnet ARES	0.99	0.99	0.99	373

Class	Precision	Recall	F1-score	Support
Brute Force	1.00	1.00	1.00	2,040
Dos/DDos	0.97	0.99	0.98	5,199
Infiltration	1.00	0.67	0.80	6
Normal	0.99	0.97	0.98	5,233
PortScan	1.00	1.00	1.00	5,077
Web Attack	1.00	0.99	1.00	413
Macro avg	0.99	0.95	0.96	18,341
Weighted avg	0.99	0.99	0.99	18,341

Table 5: Classification Performance — GAN-Augmented Model

True \ Pred	Botnet	Brute F.	Dos/DDos	Infilt.	Normal	PortScan	Web
Botnet ARES	370	0	0	0	3	0	0
Brute Force	0	2038	0	0	2	0	0
Dos/DDos	0	0	5165	0	34	0	0
Infiltration	0	0	0	4	2	0	0
Normal	3	1	131	0	5095	3	0
PortScan	0	0	2	0	0	5075	0
Web Attack	0	1	1	0	1	0	0

5.2.2 Confusion Matrix

Table 6: Confusion Matrix — GAN-Augmented Model (rows = true class, columns = predicted class)

The dominant confusion is Normal traffic being misclassified as Dos/DDos in 131 of 5,233 cases (2.5%), and the reverse (Dos/DDos as Normal) in 34 of 5,199 cases — consistent

with the two classes sharing similar high-throughput flow signatures. The only class with materially weaker recall is Infiltration (0.67, i.e. 4 of 6 test flows correctly identified), which is expected given only 29 Infiltration flows exist in the entire 91,830-row dataset before splitting.

5.2.3 Statistical Validation of the 99% Accuracy Claim

The mid-term feedback asked for a statistical comparison to substantiate the reported 99% accuracy, rather than presenting it as a single unqualified number. Two complementary analyses were added: a paired significance test against a non-augmented baseline, and bootstrap confidence intervals for both models.

Baseline Model (No GAN Augmentation)

A second XGBoost classifier was trained with identical hyperparameters (300 estimators, `max_depth=7`, `learning_rate=0.08`, `multi:softprob`) but on the original, imbalanced 73,362-row training split — i.e. with Phase 2 (GAN augmentation) skipped — and evaluated on the same 18,341-flow test set used for the GAN-augmented model. This baseline reached

98.99% accuracy, essentially matching the GAN-augmented model's 99.00%.

McNemar's Test

Because both models are evaluated on the exact same test flows, their predictions are paired, so McNemar's test is the appropriate significance test for “does GAN augmentation change which flows get classified correctly” (rather than an unpaired test, which would not use the shared test set). The contingency table of correct/incorrect predictions was:

	Baseline correct	Baseline wrong
GAN-augmented correct	18,135	22
GAN-augmented wrong	20	164

Table 7: McNemar Contingency Table (GAN-Augmented vs. Baseline)

McNemar's test (with continuity correction) on the discordant pairs (22 flows only the GAN model got right, 20 flows only the baseline got right) gives a test statistic of 0.024 and a p-value of 0.877. At any conventional significance threshold (e.g. $\alpha = 0.05$), this is not statistically significant: the data do not support a claim that GAN augmentation changed overall accuracy in this run. This is an important, honest finding — it does not mean the pipeline is broken, but it does mean the 99.00% figure should be presented as “the accuracy of the full pipeline including GAN augmentation” rather than “99% because of the GAN”; the discussion in Section 5.4 covers why, and what would need to change for the GAN stage to show a measurable benefit.

Bootstrap Confidence Intervals

To express the uncertainty in the 99.00% figure itself (rather than only comparing two models), the test set was resampled with replacement 2,000 times and accuracy recomputed on each resample, giving a 95% confidence interval of [98.85%, 99.14%] for the GAN-augmented model and [98.84%, 99.12%] for the baseline. The two intervals overlap

almost completely, which is consistent with the McNemar result above: the accuracy claim of $\approx 99\%$ is well supported by the test set, but the credit for reaching it cannot statistically be assigned to the GAN stage specifically based on overall accuracy alone.

5.2.4 GAN-Generated Data Realism Assessment

The second methodological gap raised at the mid-term defense was whether the GAN's synthetic minority-class samples are realistic, rather than assuming so because the adversarial loss converged. Two standard tests for synthetic tabular data were applied for each of the six minority classes that received GAN-generated samples: a real-vs-synthetic discriminability test (a held-out Random Forest, independent of the GAN's own discriminator, trained to tell real training samples of a class apart from that class's GAN-generated samples, scored with 3-fold cross-validated ROC-AUC — an AUC near 0.50 would mean the two are indistinguishable, i.e. realistic; an AUC near 1.00 means they are trivially separable), and a per-feature Kolmogorov–Smirnov (KS) test (comparing the distribution of real vs. synthetic values for each of the 80 processed features per class; the mean KS statistic summarizes how different the overall distributions are, from 0 = identical to 1 = fully disjoint).

Class	Real samples	Synthetic samples	Real-vs-Synth AUC	Mean KS statistic
Botnet ARES	1,491	19,441	1.000	0.605
Brute Force	8,158	12,774	1.000	0.541
Dos/DDos	20,794	138	1.000	0.630
Infiltration	23	20,909	1.000	0.610
Class	Real samples	Synthetic samples	Real-vs-Synth AUC	Mean KS statistic
PortScan	20,310	622	1.000	0.688
Web Attack	1,654	19,278	1.000	0.764

Table 8: GAN Realism — Real-vs-Synthetic AUC and KS Statistics

The results are consistent and unambiguous: for every augmented class, the discriminability AUC is 1.000 and the mean KS statistic is well above 0.5, meaning a simple classifier can perfectly separate real from synthetic flows, and the underlying feature distributions differ substantially. This is despite the GAN's own adversarial loss converging (discriminator loss near zero, generator loss increasing — Appendix A), which illustrates a well-known failure

mode of vanilla GAN training on tabular data: the discriminator can win decisively while the generator has only learned a coarse approximation of the real distribution, particularly for scaled, mixed continuous/binary features like the standardized, log-transformed flow statistics used here. Low adversarial loss is therefore not, by itself, evidence of realistic samples — which is precisely why an independent discriminability test was needed rather than reporting the GAN's own training loss.

5.3 Comparison with Objectives

Objective (Chapter 1)	Outcome
End-to-end preprocessing pipeline	Met — 91,703/91,830 flows cleaned and transformed to an 80-column matrix.
PyTorch GAN balancing layer	Partially met — implemented and trains to convergence, but Section 5.2.4 shows synthetic samples are not yet statistically realistic; accuracy benefit not significant (Section 5.2.3).
Optimized XGBoost classification	Met — 99.00% overall accuracy, macro-avg recall 0.95 on the held-out test set.
Multi-threaded Scapy sniffer, 79 features	Met — bidirectional flow reconstruction and full 79-feature schema implemented (Section 4.1.4).
Persistent storage & dashboard	Partially met — SQLite alert logging implemented; Streamlit dashboard remains as future work (Section 6.2).
Statistical validation (final-report addition)	Met — McNemar's test, bootstrap CIs, and GAN realism tests implemented and reported (Sections 5.2.3–5.2.4).

Table 9: Comparison of Project Outcomes with Stated Objectives

5.4 Discussion of Findings

The final XGBoost model reaches 99.00% overall accuracy on unseen test data, with precision and recall above 0.97 for every class except the extremely rare Infiltration class (6 test samples). Looking at per-class recall rather than only overall accuracy gives a more complete picture: the GAN-augmented model's recall on Infiltration (0.67, 4/6) is identical to the baseline's (0.6667, 4/6) in this run, and its macro-averaged recall (0.95) is close to the baseline's (0.9459). Because the three smallest classes together make up under 4% of the test set, a small number of misclassifications there moves overall accuracy only

marginally, so overall accuracy is not a sensitive enough metric to detect the GAN's effect — macro-averaged recall and per-class recall on the rare classes are the metrics recommended for tracking in any subsequent iteration of this project, in place of headline accuracy alone.

Taken together, the two new evaluation layers added for this report — the paired statistical comparison (Section 5.2.3) and the independent GAN realism check (Section 5.2.4) — are less flattering than a single headline accuracy number, but make the 99% claim considerably more defensible under questioning, because they show what was actually tested rather than only what was achieved. They also identify exactly which component (the GAN balancing stage) needs further work, and point to a concrete, testable path — WGAN-GP loss, a feature-matching/moment-matching loss term, or a single conditional GAN across all classes (a CTGAN-style architecture) in place of six independent per-class GANs — for improving it in a subsequent iteration, to be re-validated with the same AUC/KS methodology before being described as “realistic” in any future presentation of this project.

Chapter 6: Conclusion and Future Work

6.1 Conclusion

This project has delivered a functional, high-performance, AI-powered Network Intrusion Detection System that moves beyond signature matching to dynamic, flow-level pattern recognition. Combining an optimized XGBoost classifier with a multi-threaded Scapy flow-capture architecture achieves 99.00% accuracy on a held-out, stratified test split of the IDS2025 dataset, with reliable performance across live missing-data handling and automated SQLite alert logging.

Beyond the mid-term result, this final report adds the statistical and methodological rigor requested during the mid-term defense: a documented rationale for the IDS2025 dataset over UNSW-NB15 (Section 2.3.1), an explicit and evidence-based account of how the metadata-only feature design handles encrypted HTTPS traffic (Section 4.1.3), a paired statistical comparison showing the 99% figure is a robust estimate of overall accuracy even though GAN augmentation's specific contribution to it is not statistically significant in the current configuration (Section 5.2.3), and an independent realism assessment showing the GAN-generated samples are not yet statistically indistinguishable from real traffic (Section 5.2.4). Presenting these findings honestly — including the negative ones — is treated here as a strength of the evaluation rather than a weakness of the system.

6.2 Limitations

- GAN-generated minority-class samples are currently distinguishable from real samples (AUC = 1.000 across all six augmented classes) and their measured contribution to overall accuracy is not statistically significant (Section 5.2.3–5.2.4).
- Content-dependent web attacks embedded in encrypted HTTPS payloads cannot be detected by design, since the system intentionally avoids TLS termination/DPI (Section 4.1.3).
- The IDS2025 dataset, while addressing class-imbalance and labelling concerns in the CICIDS2017 family, is newer and less independently benchmarked than UNSW-

NB15, so absolute accuracy figures should not be directly compared across datasets (Section 2.3.1).

- Live-deployment load testing (packet loss and latency under saturated network links) has not yet been performed.

6.3 Future Work

- Improve GAN realism and re-run the same AUC/KS evaluation to confirm improvement, using WGAN-GP loss, a feature-matching loss term, or a single conditional (CTGAN-style) GAN across all classes.
- Replace the terminal interface with a Streamlit monitoring dashboard for real-time visualization.
- Migrate from serialized pickle files to the native `xgb_nids_engine.json` configuration to reduce live-inference latency.
- Prototype JA3/JA3S TLS fingerprinting as a decryption-free enhancement for encrypted-traffic scenarios (Section 4.1.3).
- Conduct load testing with traffic-generation utilities to measure latency and packet loss under heavy network load.

References and Bibliography

- [1] R. Panigrahi and S. Borah, “IDS2025 (Balanced Intrusion Detection Evaluation Dataset),” Mendeley Data, V1, Nov. 2025. doi: 10.17632/pkskt3fv3v.1.
- [2] R. Panigrahi and S. Borah, “A detailed analysis of CICIDS2017 dataset for designing Intrusion Detection Systems,” *Int. J. Eng. Technol.*, vol. 7, no. 3.24, pp. 479–482, 2018.
- [3] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Portugal, Jan. 2018.
- [4] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun, and A. A. Ghorbani, “Characterization of Tor Traffic Using Time Based Features,” in *Proc. 3rd Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Porto, Portugal, 2017.
- [5] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, “Characterization of Encrypted and VPN Traffic Using Time-Related Features,” in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Rome, Italy, 2016, pp. 407–414.
- [6] N. Moustafa and J. Slay, “UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set),” in *Proc. Military Commun. Inf. Syst. Conf. (MilCIS)*, 2015, pp. 1–6.
- [7] N. Moustafa and J. Slay, “The evaluation of Network Anomaly Detection Systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set,” *Inf. Secur. J.*, vol. 25, no. 1–3, pp. 18–31, 2016.
- [8] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [9] I. J. Goodfellow et al., “Generative Adversarial Networks,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2014, pp. 2672–2680.

- [10] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, “Modeling Tabular Data using Conditional GAN,” in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2019.
- [11] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [12] O. I. Ivanov, “Standard generative adversarial networks for data augmentation in imbalanced intrusion detection system datasets,” *Russian Technological Journal*, vol. 14, no. 3, pp. 7–23, Jun. 2026.
- [13] A. L. Sanders, J. Weber, and M. Collins, “Class-specific GAN-based minority data augmentation for cyberattack detection using the UWF-ZeekData22 dataset,” *MDPI Metrics*, vol. 14, no. 2, pp. 117–134, Feb. 2026.
- [14] T. N. Sharma and V. K. Joshi, “Network intrusion detection system using XGBoost,” *Distributed Machine Learning Systems Review*, vol. 22, no. 6, pp. 89–97, Jun. 2026.
- [15] R. K. Verma and S. A. Khan, “AI-based multilayer intrusion detection system for IoT networks using XGBoost edge inference,” in Proc. 2026 Int. Conf. Advances in Cyber-Security (ICACS), Kathmandu, Nepal, May 2026, pp. 412–419.
- [16] M. Engelen, T. Robic, and W. Joosen, “Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study,” in Proc. IEEE Security and Privacy Workshops (SPW), 2021, pp. 7–12.

Appendices

Appendix A: Sample Console Output

GAN training phase (excerpt — one line per minority class trained; each ran for 100 epochs with the Adam optimizer):

```
=====
PHASE 2: GAN DATA AUGMENTATION
===== [*]
Training GAN for minority class: 'Botnet ARES'...
[*] Training GAN for minority class: 'Brute Force'...
[*] Training GAN for minority class: 'Dos/DDos'...
[*] Training GAN for minority class: 'Infiltration'...
[*] Training GAN for minority class: 'PortScan'...
[*] Training GAN for minority class: 'Web Attack'...
[+] Balanced training matrices saved (146524 rows).
```

XGBoost training phase (excerpt — full classification report continues as in Table 5):

```
[*] Configuring XGBoost Hyperparameters...
[*] Training XGBoost (this may take a few minutes)...
[0]    validation_0-mlogloss:1.69386
[25]    validation_0-mlogloss:0.22771 [50]
validation_0-mlogloss:0.06735
[75]    validation_0-mlogloss:0.03862 [100]
validation_0-mlogloss:0.03104
[150]    validation_0-mlogloss:0.02747 [200]
validation_0-mlogloss:0.02725 [250]
validation_0-mlogloss:0.02774
[299]    validation_0-mlogloss:0.02836
[+] Model optimization convergence complete.
[*] Evaluating on unseen test data...

    SYSTEM VALIDATION PERFORMANCE: OVERALL ACCURACY = 99.00%
```

(Insert screenshots of the live terminal/dashboard from your own running system here for the final submission — these were reconstructed from the actual training logs produced by re-running backend/train_model.py end-to-end on the full IDS2025.csv, rather than captured as an image, so they are included as text for traceability.)